



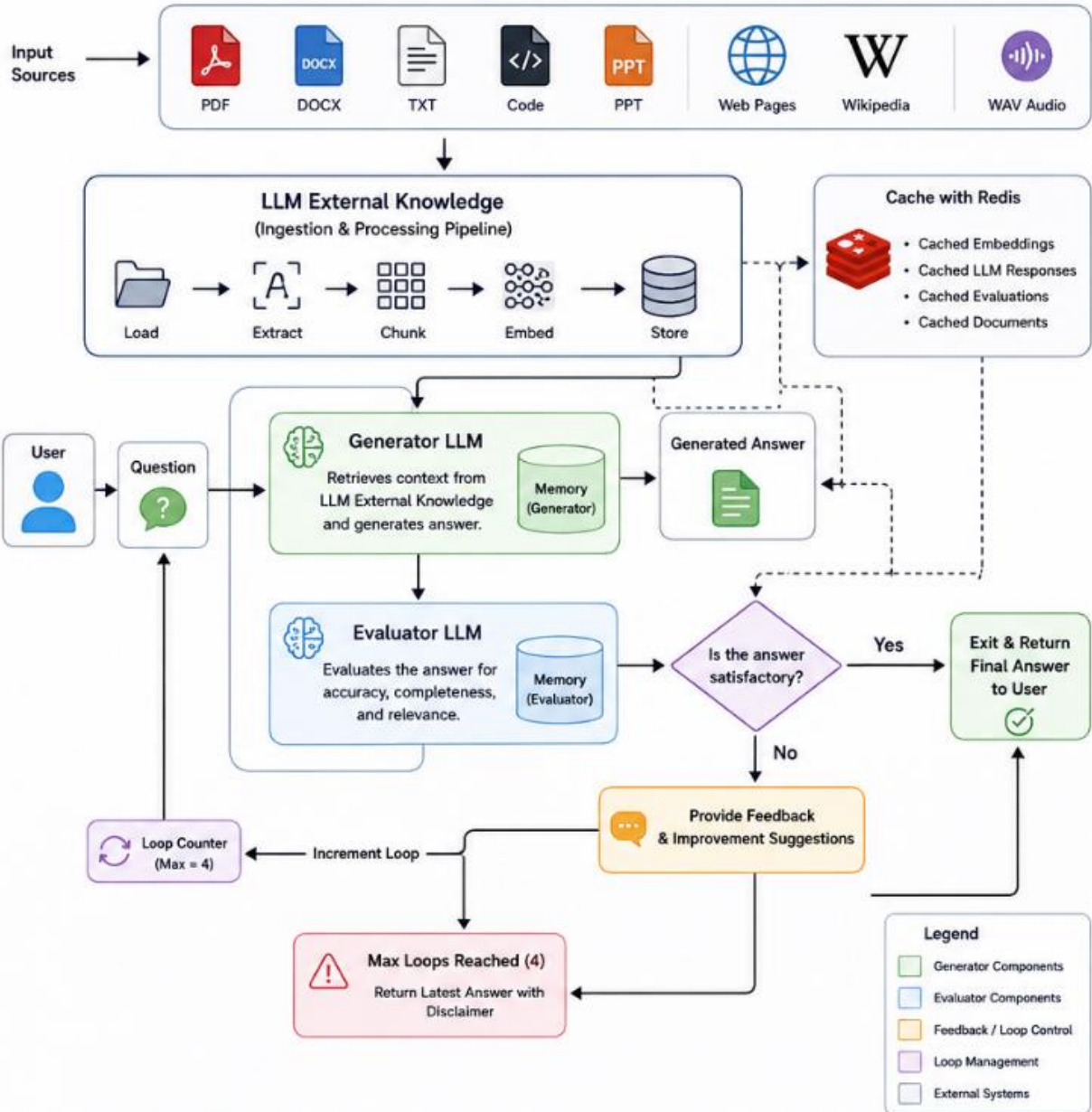
NLP INTERNSHIP

Natural Language Processing
Learn. Build. *Innovate.*



Project 3

Evaluator - Generator Workflow



- ☐ External Systems
- ☐ Loop Management
- ☐ Feedback / Loop Control
- ☐ Evaluator Components
- ☐ Generator Components

Project Overview

The goal of this task is to design and implement an intelligent AI-powered platform based on an Evaluator–Generator workflow.

The platform will allow users to provide different types of external knowledge, including documents, source code, web pages, Wikipedia pages, and audio files.

This information will be processed and made available as an external knowledge source for Large Language Models (LLMs).

The main objective is to build a reliable question-answering system that does not simply generate answers, but also evaluates its own responses and iteratively improves them when necessary.

The system will contain two specialized LLM agents a Generator LLM and an Evaluator LLM.

These two LLMs will work together in a controlled feedback loop to produce a high-quality final answer.

Supported External Knowledge

The platform should provide a flexible interface through which users can upload or provide different types of knowledge sources.

Supported inputs should include:

- PDF documents
- DOCX documents
- TXT files
- Source-code files
- PowerPoint (PPT/PPTX) presentations
- Web pages through URLs
- Wikipedia pages
- WAV audio files

All uploaded or provided sources will be processed through an ingestion pipeline before being stored in the system's LLM External Knowledge layer.

The ingestion pipeline should be responsible for loading the source, extracting its content, transforming it into an appropriate textual representation, splitting the content into meaningful chunks, generating embeddings where required, and storing the resulting knowledge in a suitable retrieval system.

For audio files such as WAV recordings, the platform should include an appropriate speech-to-text stage so that the spoken content can become searchable textual knowledge.

The purpose of this layer is to create a unified knowledge interface regardless of the original input format.

Generator LLM

The first intelligent component is the **Generator LLM**.

The Generator receives the user's question and retrieves relevant information from the LLM External Knowledge layer. It then uses the retrieved context to formulate an answer.

The Generator must follow an important grounding rule: **it should answer using the available external knowledge and should not invent information that cannot be supported by the provided sources.**

If the user's question cannot be answered using the available knowledge, the Generator should explicitly state that the required information is not available in the provided sources rather than hallucinating an answer.

The Generator will also have its own dedicated memory. This memory must be isolated from the Evaluator's memory and should store only information

relevant to the Generator's operation, such as conversation context, previous generated answers, retrieved context, and previous improvement attempts.

Evaluator LLM

The second intelligent component is the **Evaluator LLM**.

The Evaluator receives the user's original question together with the Generator's current answer and evaluates whether the answer satisfies the required quality criteria.

The evaluation should consider factors such as:

- **Accuracy**
- **Relevance to the user's question**
- **Completeness**
- **Consistency with the external knowledge**
- **Grounding in retrieved information**
- **Absence of unsupported claims**
- **Overall answer quality**

The Evaluator should produce a decision indicating whether the generated answer is acceptable or requires improvement.

If the answer is considered satisfactory, the workflow terminates and the answer is returned to the user.

If the answer is not satisfactory, the Evaluator must generate meaningful feedback explaining what should be improved. This feedback is then sent back to the Generator, which produces an improved version of the answer.

The Evaluator must also have its own dedicated memory. This memory must remain completely isolated from the Generator's memory and should contain evaluation history, previous feedback, evaluation decisions, and other information required by the Evaluator.

Evaluator–Generator Feedback Loop

The central component of the project is the iterative feedback loop.

The workflow can be represented as:

User Question → Generator → Generated Answer → Evaluator → Decision

If the Evaluator determines that the answer is acceptable:

Evaluator → Exit → Final Answer

If the Evaluator determines that the answer requires improvement:

Evaluator → Feedback → Generator → Improved Answer → Evaluator

This process continues until either:

1. The Evaluator approves the answer, or
2. The maximum number of iterations is reached.

The system must enforce a strict maximum loop count of 4 iterations to prevent infinite execution, excessive LLM usage, and unnecessary computational cost.

A loop counter should therefore be maintained as part of the workflow state.

If the maximum number of four iterations is reached without receiving an acceptable evaluation, the platform should terminate the loop and return the latest generated answer with an appropriate indication that the answer could not be fully validated.

Independent Agent Memory

Memory isolation is a mandatory architectural requirement.

The Generator and Evaluator must not share the same memory instance.

The architecture should therefore contain:

Generator → Generator Memory

and separately:

Evaluator → Evaluator Memory

This separation is important because the two agents have fundamentally different responsibilities. The Generator is responsible for producing answers, while the Evaluator is responsible for judging and improving those answers.

The system should prevent accidental cross-access between the two memory stores.

Redis Caching Layer

The platform must include Redis as a caching layer.

Redis should be used to reduce unnecessary computation and improve system performance.

Potential caching targets include processed documents, embeddings, frequently requested retrieval results, repeated questions, LLM responses, and evaluation results where appropriate.

A simplified architecture is:

User → Application → Redis Cache → Knowledge
Retrieval / LLM

When an identical or equivalent request can safely use a previously computed result, the system should retrieve the information from Redis instead of unnecessarily executing the same expensive operation again.

Caching should be designed carefully so that stale or inappropriate information is not returned when the underlying knowledge or workflow state has changed.

Expected Deliverable

The final deliverable should be a functional platform demonstrating the complete Evaluator–Generator architecture.

The implementation should include:

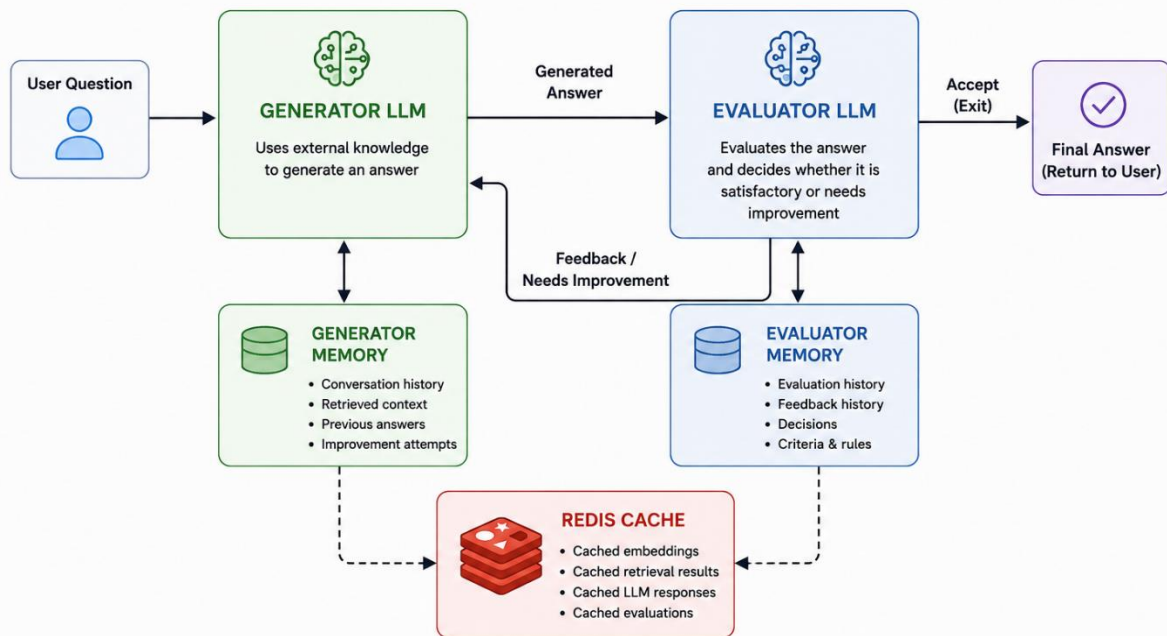
- A user interface for uploading files and providing URLs.
- Support for all specified knowledge formats.
- An external knowledge ingestion and retrieval pipeline.
- A Generator LLM.
- An independent Generator memory.
- An Evaluator LLM.
- An independent Evaluator memory.
- An iterative feedback mechanism.
- A maximum loop count of four.
- Redis caching.
- LangChain Chains.
- LCEL-based workflow orchestration.
- Clear handling of unknown or unsupported questions.
- Error handling and validation.

- Logging and monitoring where appropriate.
- Documentation explaining the architecture and workflow.

The system should ultimately demonstrate that an LLM application can move beyond simple one-shot generation toward a self-evaluating and iterative answer-generation architecture. By combining external knowledge, retrieval, independent Generator and Evaluator agents, isolated memories, Redis caching, and LCEL-based orchestration, the platform should provide answers that are more grounded, reliable, and easier to validate than a conventional single-LLM question-answering system.

The generated workflow diagram should be used as the architectural reference for the implementation.

Evaluator – Generator Workflow



Iterative loop continues until answer is accepted or max iterations (4) is reached.